Day 6 – System Stabilization, Visualization, and Documentation

1. Project Status Overview

Day 6 focused on improving the overall quality, presentation, and organization of the VisionGuard Phase 1 system. After completing real Arduino hardware integration on Day 5, today's work was centered on system validation, visual enhancement, documentation improvement, and preparing the project for professional presentation.

The main goal was to make the project not only functional, but also **clean, demonstrable, and report-ready**.

---

2. Full System Re-Test

A complete end-to-end test was performed to verify that all major modules are working together correctly.

Tested Workflow

```
Webcam Input
→ Motion Detection
→ Face Detection
→ Decision Logic
→ Python Serial Communication
→ Arduino Uno
→ LED + Buzzer Alert
```

Test Scenarios

Test Case	Expected Result	Status
No motion	No alert	Passed
Motion only	No alert	Passed
Face only	No alert	Passed
Motion + face	LED and buzzer ON	Passed
Intruder leaves	LED and buzzer OFF	Passed

Result

The complete VisionGuard Phase 1 system was successfully re-tested. The AI detection logic, Arduino communication, LED alert, and buzzer alert worked as expected.

3. Motion Area Visualization Added

A new visual enhancement was added to display the movement area on the live video feed.

Earlier, the system only displayed:

```
Faces count
Motion status
SAFE / ALERT status
Face bounding box
```

Today, the system was upgraded to show a **motion bounding box** around the detected moving region.

Motion Visualization Code

```
# Find moving areas from threshold image
contours, _ = cv2.findContours(
    thresh,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

# Draw only the largest motion area to avoid too many boxes
if contours:
    largest_contour = max(contours, key=cv2.contourArea)

    if cv2.contourArea(largest_contour) > 3000:
        x, y, w, h = cv2.boundingRect(largest_contour)

        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            (255, 0, 0),
            2
        )

        cv2.putText(
            frame,
            "Motion",
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (255, 0, 0),
            2
        )
```

Improvement Made

Initially, too many motion boxes appeared due to small noise regions being detected as separate contours. To solve this, the logic was improved to display **only the largest meaningful motion area**.

This made the visual output cleaner and more suitable for demonstration.

---

4. Prototype Photo Placement in Report

The hardware prototype photo was added to the Day 5 technical report in the correct section.

Placement

```
Day 5 Report
→ Section 3: LED and Buzzer Hardware Setup
→ After 3.1 LED Wiring and 3.2 Buzzer Wiring
→ Before 3.3 Hardware Test Code
```

Caption Used

```
Figure 5.1: Prototype Circuit Connection – Arduino Uno with LED and Buzzer
```

Additional Demo Photo Caption

For the photo showing both the circuit and the intruder detection output, the following caption was prepared:

```
Figure 5.2: VisionGuard Prototype Demonstration – Arduino Alert Circuit
with Real-Time Intruder Detection Output
```

Description

```
The figure shows the complete VisionGuard prototype during testing. The
Arduino Uno R3 hardware circuit with LED and buzzer is connected to the
Python-based AI intruder detection system. When motion and face detection
conditions are satisfied, the system triggers the physical alert and
displays the real-time detection output on screen.
```

This improved the technical report by adding visual proof of hardware implementation and system testing.

---

5. Project Folder Cleanup

The project directory was organized into a clean professional structure for future GitHub upload and report submission.

Final Folder Structure

```
VisionGuard/
│
├── code/
│   ├── visionguard_main.py
│   └── arduino_test.py
│
├── arduino/
│   ├── command_mode.ino
│   └── led_buzzer_test.ino
│
├── captures/
│   ├── sample_intruder_capture.jpg
│
├── docs/
│   ├── Day_5_Technical_Report
│   └── VisionGuard_Report.docx
│
├── images/
│   ├── prototype_circuit.jpg
│   ├── complete_demo.jpg
│   ├── tinkercad_circuit.png
│   └── phase_1_architecture.png
│
├── README.md
├── requirements.txt
└── .gitignore
```

Purpose of Cleanup

The cleanup ensures:

```
Python code is separated from Arduino code
Reports are stored in documentation folder
Images and prototype photos are organized
Captured evidence images are separated
GitHub upload becomes easier
Project looks professional and understandable
```

---

6. README Draft Completed

A basic `README.md` file was created for the VisionGuard project.

README Content Includes

```
Project title
Short project description
Features
Hardware used
Software used
System workflow
Output behavior
Future scope
```

README Draft

```
# VisionGuard – AI Intruder Detection System

VisionGuard is an AI + Embedded Systems project that detects intruders
using real-time motion detection and face detection. When an intruder is
detected, Python sends a serial command to Arduino Uno, which triggers an
LED and buzzer alert.

## Features

- Real-time webcam monitoring
- Motion detection using frame differencing
- Face detection using OpenCV Haar Cascade
- Motion memory for stable detection
- Arduino Uno LED and buzzer alert
- Cooldown-based image capture
- Real-time status display
- Motion area visualization

## Hardware Used

- Arduino Uno R3
- LED
- 220Ω resistor
- Active buzzer
- Breadboard
- Jumper wires
- Laptop webcam

## Software Used

- Python
- OpenCV
- PySerial
- Arduino IDE

## System Workflow

Camera → Motion Detection → Face Detection → Decision Logic → Python Serial
Communication → Arduino Alert
```

```
## Output

When motion and face detection conditions are satisfied:

- LED turns ON
- Buzzer turns ON
- Intruder image is saved in the captures folder
- System status changes to ALERT

## Future Scope

- ESP32 WiFi alert system
- Telegram or email notifications
- Authorized vs unauthorized face recognition
- Cloud storage for captured images
```

---

7. Day 6 Key Achievements

```
Full VisionGuard system re-tested successfully
Motion area visualization added
Multiple motion boxes issue solved
Prototype circuit photo placement completed
Complete demo photo caption prepared
Project folder cleaned and organized
README draft completed
GitHub-ready structure planned
```

---

8. Technical Learning Outcomes

Day 6 helped strengthen skills in:

```
System testing
Visual debugging
Contour-based motion visualization
Project documentation
Report image placement
Folder organization
README writing
GitHub preparation
Professional project presentation
```

---

9. Development Conclusion

Day 6 transformed VisionGuard from a working prototype into a more polished and presentable engineering project.

The system now has:

```
Functional AI detection
Real Arduino hardware alert
Motion visualization
Organized documentation
Clean project structure
README file for presentation
```

This makes VisionGuard Phase 1 suitable for academic submission, GitHub upload, and portfolio presentation.

---

10. Next Development Roadmap

The next development steps are:

```
1. Final code cleanup
2. Prepare GitHub upload
3. Add requirements.txt and .gitignore
4. Improve README with images and architecture diagram
5. Begin Phase 2 using ESP32 for IoT-based alerts
```